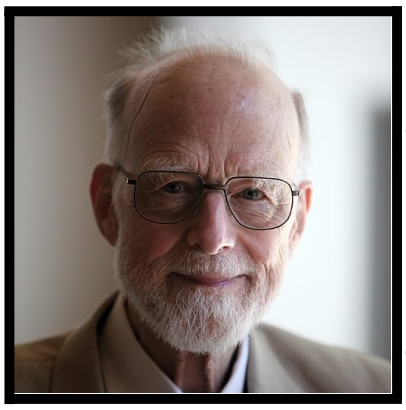
PARTIE SACHA

1. Qu'est-ce que null et la NullPointerException ?

L’histoire de la valeur nulle



En Java, `null` représente une référence vide, c'est-à-dire l'absence de valeur ou le fait qu'une variable ne pointe vers aucun objet en mémoire.

Introduit par Tony Hoare en 1965 dans le langage ALGOL W, il qualifia lui-même cette invention de « *billion-dollar mistake* » (l'erreur à un milliard de dollars) en raison du nombre incalculable d’erreurs, de vulnérabilités et de pannes système qu'elle a engendrés au fil des décennies.

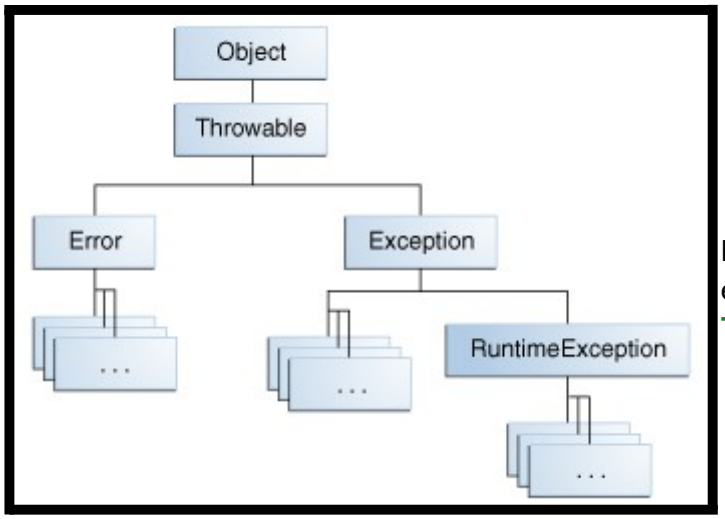
Errors, exceptions (checked, unchecked), et NullPointerException

Il est important de savoir qu’une application peut rencontrer deux grands types de problèmes:

Les **erreurs de compilation** qui sont détectables par l'IDE avant même de lancer le programme

Les **erreurs à l'exécution sont** des situations anormales survenant pendant que le programme tourne. Elle est levée par la machine virtuelle **(JVM)**, et cela peut inclure par exemple une division par zéro ou, dans le cas de `NullPointerException`, lorsqu'un programme tente d'utiliser une référence qui pointe vers null.

Comparées aux **Error** qui sont de graves **erreurs fatales** à l’exécution de l’application (qui sont des erreurs internes ou de manque de ressources comme NoSuchMethodError ou OutOfMemoryError), les **Exception** sont des **erreurs récupérables** qui peuvent être attrapées et traitées afin de poursuivre proprement l’exécution de l'application.



En Java, toutes les erreurs et exceptions héritent de la classe mère **Throwable**

`NullPointerException` fait partie de la famille des **Unchecked Exception** (ou d'exceptions non contrôlés). Comparé aux **Checked Exception,** ils ne sont visibles qu'au moment de l'exécution et hérite de RunTimeException.

2. Les différentes manières de traiter un `null` en Java

Face au risque de `NullPointerException`, il y a plusieurs manières de gérer les valeurs nulles. La méthode la plus ancienne et la plus basique consiste à tester **si la variable est différente de** `null` avant de l'utiliser.

```
if (utilisateur != null) {
    Adresse adr = utilisateur.getAdresse();
    if (adr != null) {
        System.out.println(adr.getVille());
    }
}
```

C'est une implémentation simple à comprendre et universelle, elle conduit rapidement à des structures de code imbriquées, ce qui rend le code verbeux et difficile à maintenir.

Cependant nous avons vu d'autres manières de traiter les exceptions, et ainsi les valeurs nulles en Java :

Nous avons d'abord le **déclenchement** explicite d'une exception avec le mot-clé `throw` (pour lever une erreur).

```
if (utilisateur == null) {
    throw new IllegalArgumentException("L'utilisateur
ne peut pas être null !");
}
```

Ensuite, la **propagation** en l'ajoutant dans la signature de la méthode avec `throws`. Cela délègue la responsabilité du traitement à la méthode appelante de niveau supérieur. Cependant, jamais de `throws` dans la signature d'un `main` car le programme plante à l'exécution : le `main` est le tout dernier endroit où il faut impérativement attraper les exceptions.

```
if (utilisateur == null) {
    throw new IllegalArgumentException("L'utilisateur
ne peut pas être null !");
}
```

Enfin, la gestion de l'exception en la **capturant** et la **traitant** (**try...catch**). Pour cela, on englobe l'instruction à risque dans un bloc try et l'on intercepte l'erreur dans un bloc catch. Une fois capturée, on peut exécuter une action de secours et afficher proprement le message d'erreur porté par l'exception grâce à **e.getMessage()** :

```
if (utilisateur == null) {
    throw new IllegalArgumentException("L'utilisateur
ne peut pas être null !");
}
```

Cependant, une alternative moderne existe et permet d'éviter l'utilisation de null en général.

3. L'alternative moderne : L'API Optional<T> (Java 8)

Qu'est-ce que `Optional<T>` et pourquoi a-t-il été introduit ?

Introduite à partir de Java 8, la classe `java.util.Optional<T>` est un conteneur qui peut soit contenir une valeur non nulle, soit être vide (`Optional.empty()` pour dire qu'il n'y a pas de valeur). L'objectif principal est d'éviter l'utilisation explicite de `null`, ce qui permet d'éliminer le besoin de le traiter, et ainsi les risques de `NullPointerException`.

D'après la documentation officielle de Java et de la communauté, `Optional` s'insère dans la signature des méthodes ou des attributs pour indiquer qu'un retour ou objet d'un certain type peut tout simplement être absent, évitant d'avoir à retourner arbitrairement `null` ou à utiliser des magic numbers (tels que `-1`) lors de l'initialisation d'un attribut.

Comment l'utiliser concrètement ? (Exemples d'utilisation)

L'utilisation de `Optional` repose sur plusieurs méthodes clés de sa Javadoc :

Créer un `Optional` :

`Optional.empty()` **:** Crée un conteneur vide.

`Optional.of(valeur)` **:** Enveloppe une valeur (lève une `NullPointerException` si la valeur passée est `null`).

`Optional.ofNullable(valeur)` **:** Crée un `Optional` vide si la valeur est `null`, ou l'enveloppe si elle est présente.

Vérifier et extraire la valeur :

`isPresent()` **:** Retourne `true` si une valeur est présente, sinon `false`

`get()` **:** Récupère la valeur contenue (attention, lève une exception si l'Optional est vide).

`orElse(valeurParDefaut)` : Permet de fournir une alternative si le conteneur est vide.

Sources :

TP de l'années dernière sur les exceptions et de cette année sur Mockito
https://en.wikipedia.org/wiki/Null_pointer
https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/NullPointerException.html
https://www.developpez.com/actu/134958/Java-apprendre-a-utiliser-la-classe-Optional-lt-T-gt-pour-eviter-d-utiliser-explicitement-null-par-Gugelhupf/
https://www.touilleur-express.fr/2014/11/07/optional-en-java-8
https://docs.oracle.com/javase/8/docs/api/java/util/Optional.html
https://en.wikipedia.org/wiki/Tony_Hoare
https://docs.oracle.com/javase/tutorial/essential/exceptions/throwing.html

IAgraphie (Gemini 3.5 Flash-Lite)

Génération de contenu : Rédaction d'une première ébauche de l'oral sur les thèmes abordés avec citation de sources (utilisation de documentation officiel, sources populaire type wikipedia et concepts TP) et exemples.

Structuration : Aide à l'organisation de la présentation et de la mise en forme du contenu pour les diapositives (slides).

Correction et révision : Clarification technique, correction de l'oral et des diapositives

PARTIE YASSINE

1. L'API `Optional<T>`

A. Un exemple inédit : La recherche en base de données

Pour bien comprendre la puissance d'`Optional`, prenons un exemple très courant qu'on a tous rencontré : la recherche d'un utilisateur en base de données. Avant, on renvoyait`null` si l'utilisateur n'existait pas, ce qui causait des plantages si on oubliait de vérifier. Avec `Optional`, la méthode est beaucoup plus explicite :

```java
public Optional<Utilisateur> findUtilisateurById(int id) {
    Utilisateur user = database.get(id);
    return Optional.ofNullable(user);
}
```

Cela force le développeur qui appelle cette méthode à gérer le cas où l'utilisateur est absent, s'inscrivant ainsi dans une approche de programmation défensive.

B. Détail de l'utilisation d'Optional avec l'exemple de Mockito (Kata Dice)

Pour reprendre l'exemple du TP Mockito vu en cours, voici comment la classe`Player` a été modifiée pour intégrer `Optional`. L'attribut `lastValue`, qui représentait l'absence de lancer avec la valeur arbitraire `-1`, utilise désormais un `Optional<Integer>` :

```java
import java.util.Optional;

public class Player {

    private String name;
    private Optional<Integer> lastValue;

    public Player(String name) {
        this.name = name;
        this.lastValue = Optional.empty();
    }

    public void play(Dice dice) {
        this.lastValue = Optional.of(dice.roll());
    }

    public Optional<Integer> getLastValue() {
        return lastValue;
    }

    public String getName() {
        return name;
    }
}
```

Du côté des tests unitaires, l'utilisation d'`Optional` change la donne. Au lieu de faire des assertions sur une valeur arbitraire comme `-1`, l'intention devient limpide : on vérifie explicitement si la valeur est présente ou non. C'est tout l'intérêt de cette deuxième capture :

```java
import static org.junit.jupiter.api.Assertions.assertFalse;

class PlayerTest {

    @Test
    void playerWithoutPlay_shouldHaveEmptyLastValue() {
        Player player = new Player("Bob");

        assertFalse(player.getLastValue().isPresent());
    }
}
```

---

2. La méthode statique `Objects.requireNonNull`

Bien que `Optional` soit parfait pour les valeurs de retour et certains attributs, il n'est pas conçu ni recommandé pour vérifier les paramètres passés à une méthode. Pour valider qu'un paramètre obligatoire n'est pas `null` lors de son affectation, on utilise plutôt la classe utilitaire `Objects`, et plus précisément sa méthode `requireNonNull`.

A. Explication et utilisation générale

La méthode statique `Objects.requireNonNull(T obj, String message)` vérifie si l'objet passé en paramètre est `null`. Si c'est le cas, elle lève immédiatement une `NullPointerException` avec le message spécifié en second paramètre. Si l'objet n'est pas null, elle le retourne.

L'objectif ici est d'appliquer le principe du **"Fail-Fast"** (échouer rapidement). Au lieu d'accepter un objet `null` silencieusement, ce qui causerait un plantage complexe à déboguer beaucoup plus loin dans le programme, on bloque le problème dès l'entrée de la méthode ou du constructeur.

Exemple d'utilisation standard :

```java
import java.util.Objects;

public class Utilisateur {

    private String pseudo;

    public Utilisateur(String pseudo) {
        this.pseudo = Objects.requireNonNull(pseudo, "Le pseudo ne peut pas être null");
    }
}
```

---

3. Lier `Objects.requireNonNull` et la notion de `record`

Pour faire le lien avec une notion plus récente vue en cours, examinons l'utilisation de cette méthode avec les records. Les records étant immuables, il est nécessaire de vérifier la validité des paramètres dès l'instanciation, même si les attributs ne sont pas explicitement déclarés.

A. Qu'est-ce qu'un record en Java ?

Introduits dans les versions récentes de Java, les `records` sont des classes de données immuables dont la syntaxe est extrêmement concise. En une seule ligne, le compilateur génère automatiquement le constructeur, les accesseurs en lecture seule, ainsi que les méthodes `equals`, `hashCode` et `toString`.

B. Utilisation conjointe dans un constructeur compact

C'est ici qu'intervient le constructeur compact. Il permet de valider les données sans avoir à réécrire les affectations classiques. En le combinant avec `requireNonNull`, on obtient un code propre et sécurisé.

Un `record` ne permet pas de modifier ses attributs une fois l'objet créé (immuabilité). Il est donc indispensable de s'assurer de la validité des données dès leur instanciation. Le constructeur compact permet d'ajouter de la logique de validation sans avoir à réécrire les affectations fastidieuses.

Exemple concret :

```java
import java.util.Objects;

public record UtilisateurRecord(String pseudo, String email) {

    public UtilisateurRecord {
        Objects.requireNonNull(pseudo, "Le pseudo est obligatoire");
        Objects.requireNonNull(email, "L'email est obligatoire");

    }
}
```

En associant `record` et `requireNonNull`, on garantit non seulement l'immuabilité, mais aussi la validation stricte des données dès la création de l'objet. Ainsi, un objet de transfert de données (DTO) ne transportera jamais de références nulles illégales.

PARTIE NATHAN

Sources :

Wikipedia, « Null object pattern » : https://en.wikipedia.org/wiki/Null_object_pattern

Refactoring Guru, « Introduce Null Object » : https://refactoring.guru/introduce-null-object

Martin Fowler, « Special Case » : https://martinfowler.com/eaaCatalog/specialCase.html

Cours R2.01 (canards, pattern Stratégie) et cours R3.04 (patterns State et Stratégie)

LE PATTERN NULL OBJECT

Nom / type
Null Object (appelé « Special Case » par Martin Fowler). Pattern de conception qui n'est pas un pattern du GoF.

Problématique
Comment éviter de multiplier les tests null dans le code client et traiter l'absence d'objet de façon cohérente ?

Intention
Fournir un objet de substitution qui implémente la même interface que les objets réels, avec un comportement neutre, pour remplacer null.

Besoin (exemple fil rouge : éditeur d'un jeu vidéo dans un CSV de ventes)

- Modélisation de départ : Jeu possède un Editeur, qui vaut null si non renseigné.
- Problèmes : test != null dupliqué dans tout le code client, risque de NullPointerException en cas d'oubli, comportement incohérent d'un endroit à l'autre.

Solution

- Interface Editeur avec la méthode getNom().
- EditeurReel : renvoie le vrai nom.
- EditeurInconnu (le Null Object) : renvoie « Inconnu ».
- Code client : jeu.getEditeur().getNom(), sans aucun if.

Classes participantes

- AbstractObject : contrat utilisé par le client. Dans l'exemple : Editeur.
- RealObject : implémentation avec le vrai comportement. Dans l'exemple : EditeurReel.
- NullObject : même contrat, comportement neutre et sans effet de bord. Dans l'exemple : EditeurInconnu.
- Client : utilise AbstractObject sans savoir quelle implémentation il manipule. Dans l'exemple : le code qui parcourt les jeux.

Bonnes pratiques

- Une seule instance du Null Object (singleton), car il n'a pas d'état.
- Éviter une méthode isNull(), qui réintroduit un test.

Avantages

- Code client plus simple et plus lisible.
- Comportement par défaut centralisé, donc cohérent.
- Repose sur le polymorphisme.

Limites

- Peut masquer des bugs : l'absence devient silencieuse.
- Pas toujours de comportement neutre pertinent (un prix inconnu à 0 serait faux).
- Classes supplémentaires à écrire et à maintenir.
- Inefficace si null est renvoyé ailleurs dans le code.

Mise en relation avec d'autres patterns

Stratégie

- Intention : encapsuler une famille d'algorithmes interchangeables.
- Structure : Contexte → interface + implémentations (même structure que Null Object).
- Lien : un Null Object est souvent une stratégie qui ne fait rien (ex. FlyNoWay dans les canards de R2.01).
- Différence : Stratégie sert à choisir un comportement, Null Object remplace l'absence d'objet.

État

- Intention : changer le comportement d'un objet selon son état interne.
- Structure : Contexte → interface + implémentations (structure proche).
- Lien : un Null Object peut être un état « vide » ou « inactif » (ex. lecteur sans morceau chargé).
- Différence : dans État, les états changent au cours du temps ; le Null Object est généralement fixe.

Optional ou Null Object ?

- Optional signale l'absence et oblige l'appelant à la traiter.
- Null Object cache l'absence derrière un comportement par défaut.

Live coding
Refactoring du code Jeu / Editeur : passage de null + if à Editeur, EditeurReel, EditeurInconnu, puis rétro-conception comparée au diagramme générique du Null Object.